

MARGINALIA

A session-aware study companion that reads beside you

PRODUCT REQUIREMENTS DOCUMENT · BASE VERSION (v1) · DRAFT · MONDAY, SEPTEMBER 28, 2026

PRODUCT	Marginalia (working title)
DOCUMENT	PRD — v1 base scope, single user, local-first
STATUS	Draft for build
PLATFORM	Local web app in a desktop browser; Node server on localhost; Claude via subscription or API key
DEFAULT MODEL	Claude Sonnet 5 for tutoring; Haiku 4.5 for cheap utility calls; Opus 5.5 on explicit request

Marginalia is a study workspace with the PDF on one side and a tutor on the other. The tutor sees the page you are on and what you have covered in this session, teaches in a style you configure for each subject, and keeps a small, editable model of what you know across sessions. It is deliberately *not* a whole-book retrieval system: the context is your reading trail, not the document.

“The book stays open. The tutor sits beside it. Nothing makes you leave the page.”

Contents

01	Problem and vision	10	Data model
02	Product principles	11	Tutor prompt design
03	Scope: v1 base, later, and non-goals	12	MCP server for Claude Code
04	Use scenarios	13	Project structure and modularity
05	Functional requirements	14	Milestones and acceptance criteria
06	UX and UI design	15	Success metrics
07	System architecture	16	Risks and open questions
08	Context engineering and usage optimization	17	Appendix: shortcuts and glossary
09	Model configuration		

SECTION 01 · WHY THIS EXISTS

Problem and vision

The problem

Studying from PDFs and books breaks down on friction, not on difficulty. A doubt means leaving the page: searching the concept, switching tabs, opening an LLM chat, pasting context, and re-explaining where you are. Each switch costs focus, and enough of them make you give up on the session.

Existing tools each cover one slice of the problem:

- **Upload-and-generate apps** digest the whole book and output summaries, flashcards and quizzes. They do not follow your reading.
- **General study modes** (ChatGPT Study Mode, Gemini Guided Learning, Claude Learning mode) teach well in a chat, but do not know which page you are on or what you covered an hour ago.
- **Page-aware side panels** (browser and PDF-reader assistants) see the page but have no pedagogy and no memory of you.

The vision

A single reading screen where a tutor follows your position and trail through the text, teaches the way a good human tutor would for that subject, opens and closes each session with you, remembers what you found hard, and plans the week with you. It works on three timescales:

TIMESCALE	ROLE	WHAT IT DOES
Minutes	Session tutor	Answers doubts about the current page, gives hints, checks understanding, challenges arguments, in the subject's style.
Days / week	Coach	Holds weekly targets per subject, shows progress, proposes next steps from session debriefs.
Months	Learner model	Tracks concepts (introduced, shaky, solid), misconceptions, open questions, and your stated preferences.

SECTION 02 · DESIGN RULES THAT SETTLE ARGUMENTS

Product principles

#	PRINCIPLE	WHAT IT MEANS IN PRACTICE
1	Session, not book	Context is the current page, the selection, the session trail and a learner snapshot. No whole-book ingestion or embeddings in v1.
2	Tutor, not answer engine	Each subject has a pedagogy and an answer policy. Hints come before solutions where configured.
3	Never break focus	Every action is reachable from the reading screen and from the keyboard. No modal flows while reading.
4	Frugal by design	Every model call has a purpose and a token budget. Passive events (page turns, highlights) never trigger calls.
5	Modular	Panes, tutor profiles, quick actions, models, OCR engines and LLM providers are registries or adapters, swappable without rewrites.
6	Local and owned	All data in one SQLite file plus the PDFs on disk. Notes export to Markdown. Nothing leaves the machine except model calls.
7	Reviewable memory	The tutor proposes; you accept. Debriefs and concept updates are shown before they are saved and remain editable.

SECTION 03 · KEEPING v1 SMALL

Scope: v1 base, later, and non-goals

P0 means required for v1. **P1** means in v1 if time allows, otherwise first follow-up. Everything in the “Later” column is explicitly out of v1.

AREA	V1 BASE	LATER
Reader	PDF.js reader, highlights, notes on highlights, page and dwell tracking, outline, search	EPUB and web articles, ink/handwriting
Text extraction	Native text layer; OCR once on import (local Tesseract) or Claude vision per page, cached	Automatic per-page engine choice, math-aware converter
Tutor	Chat, quick actions, highlight-to-ask, answer policies, go-deeper escalation, marginalia pins	Voice, richer proactive behaviour
Sessions	Open, during and close ritual; session types; structured debrief	Auto-scheduling from calendar
Learner model	Concepts with status and evidence, open questions, stated preferences	Cross-subject concept linking
Notes and questions	Own and generated notes (3 compression levels), parked questions	Note graph, backlinks
Practice	Generate items from covered pages; simple review queue (P1)	Tuned spaced repetition, exam mode
Planning	Weekly targets per subject with locally computed progress (P1)	Adaptive coach that rewrites plans
Integrations	MCP server for Claude Code (P1)	Anki export, sync across devices
Models	Per-role model settings, subscription or API provider, usage meter, lean mode	Non-Claude providers

Non-goals for v1

- Accounts, multi-user, cloud sync or hosting.
- Mobile layouts (desktop browser only).
- Whole-book RAG, vector databases or embedding pipelines.
- Audio overviews, podcasts, video, slide generation.
- Gamification: streaks, XP, badges.
- Installer or desktop packaging; v1 runs with one terminal command.

SECTION 04 · ONE USER, FOUR KINDS OF STUDY

Use scenarios

The only user is the builder: a CS student who self-studies mathematics, neuroscience, philosophy and computer systems. The scenarios below define how the tutor must behave differently per subject.

Analysis, working through Spivak

Problem-solving session on continuity. Stuck on a step in a proof, you select the line and press **Hint**. The tutor gives the smallest nudge that unblocks you (“What does the definition give you if you choose δ for $\epsilon/2$?”) and does not write the proof. You park a question about uniform continuity. At close, the debrief marks “ ϵ – δ manipulation” as shaky and suggests two exercises for next time.

Neuroscience, first read

The tutor explains unfamiliar terms in plain language first, uses analogies, links back to earlier pages you read, and ends explanations with one check question. At close you request condensed notes of the session.

Philosophy, critical reading

The tutor acts as an interlocutor. It asks you to reconstruct the argument as premises and conclusion, raises objections, flags equivocation, and asks you to steelman the view you disagree with. It avoids telling you what to conclude.

Computer systems, mechanisms

The tutor walks through mechanisms step by step and asks prediction questions (“What happens to the TLB on a context switch?”) before revealing the answer, tying ideas to real code or hardware.

SECTION 05 · WHAT v1 MUST DO

Functional requirements

5.1 Library and import

ID	PRI	REQUIREMENT
LIB-1	P0	Import a PDF by file picker or drag-and-drop. Copy it into the data directory and hash it to prevent duplicates.
LIB-2	P0	Assign the book to a subject at import; changeable later.
LIB-3	P0	Detect a usable text layer per page (character-count threshold). Mark pages that need OCR.
LIB-4	P0	Per-book text source: <i>Native</i> , <i>Local OCR</i> (ocrmypdf + Tesseract, produces a searchable copy once), or <i>Vision</i> (page image read by the vision model on first visit, then cached). Never OCR the same page twice.
LIB-5	P1	Read the PDF outline to label pages with chapter and section. If none exists, allow manual chapter ranges.
LIB-6	P1	Page offset so printed page numbers match the book (p. 143 in the book, not PDF index 161).
LIB-7	P0	Library lists subjects, then books, with last page read, progress, and last session date.

5.2 Reader

ID	PRI	REQUIREMENT
RD-1	P0	Render with PDF.js: continuous scroll, zoom, fit-width. Keeps working when models are unavailable.
RD-2	P0	Select text; highlight in three print-friendly styles (solid, underline, margin bracket); attach a note to a highlight.
RD-3	P0	Track the current page (the page at least half visible) and dwell time. Log events locally, debounced. No model calls.
RD-4	P0	Outline sidebar, jump to page, search within the book using the cached text.
RD-5	P1	Marginalia: tutor replies pinned to a page appear as numbered notes in the page margin; clicking opens the thread.
RD-6	P1	Focus mode: only the PDF and a one-line tutor input remain visible.

5.3 Tutor

ID	PRI	REQUIREMENT
TU-1	P0	Chat panel bound to the active session. Streamed replies. Markdown and LaTeX (KaTeX) rendering.
TU-2	P0	Each turn is built by the context builder (Section 8): profile, learner snapshot, session header, trail summary, recent turns, current page, selection.
TU-3	P0	Highlight-to-ask: a selection offers Ask, Explain and Hint. The selection is quoted in the message with its page reference.
TU-4	P0	Quick actions: Explain , Hint (smallest useful nudge), Check me (tutor asks you to explain back), Challenge (objection or counterexample), Summarize page . Each is a registered action with its own short prompt and model role.
TU-5	P0	The subject’s answer policy is enforced, e.g. hints-first; a full solution requires an explicit “reveal”.
TU-6	P1	Go deeper : re-ask the current question with the deep model. Only ever on explicit request.

ID	PRI	REQUIREMENT
TU-7	P1	One click to pin a reply to its page (marginalia) or save it as a note.
TU-8	P1	Page-image mode for equation-heavy or scanned pages: attach a downscaled render of the page, per message or per book.
TU-9	P0	Stop, regenerate, and edit-and-resend the last message.

5.4 Subjects and tutor profiles

ID	PRI	REQUIREMENT
TP-1	P0	Each subject has a tutor profile: name, persona, pedagogy style (Socratic, Explainer, Interlocutor, Coach), answer policy (hints-first or explain-first), verbosity, notation preferences, custom rules, model overrides.
TP-2	P0	Four presets ship: Analysis, Neuroscience, Philosophy, Systems (Section 11).
TP-3	P1	Per-session-type overrides, e.g. Review sessions quiz more and explain less.
TP-4	P0	Profiles are data (JSON), edited in a form, importable and exportable as files.

5.5 Sessions

ID	PRI	REQUIREMENT
SE-1	P0	Start a session: choose book, type (First read, Problem solving, Review), optional goal and time box. The layout preset follows the type.
SE-2	P0	Opening: the tutor states where you left off, open questions for this chapter, and relevant shaky concepts. Templated from local data in lean mode; one fast-model call otherwise.
SE-3	P0	During: log trail events (page view with dwell, highlight, note, parked question, message, action).
SE-4	P0	Close: one structured call returns a debrief (summary, covered pages, concept updates, misconceptions, open questions, next step, optional notes draft). You review and accept or edit before it is saved.
SE-5	P0	After 30 minutes of inactivity the session is marked for closing. The debrief is generated when you next open the app, never silently in the background.
SE-6	P1	Session history: date, duration, pages, debrief.

5.6 Notes

ID	PRI	REQUIREMENT
N0-1	P0	Notes in Markdown with LaTeX, attached to a book, page range, session or concept. Written by you or generated.
N0-2	P0	Make notes on a page range or a session at a chosen compression: Outline, Condensed, or Full. Generated only on request or accepted from a debrief.
N0-3	P1	Notes pane filtered by book, chapter or concept; export all notes to a Markdown folder.

5.7 Parked questions

ID	PRI	REQUIREMENT
QU-1	P0	Park a question in one keystroke from anywhere; page reference and selection attach automatically.
QU-2	P0	States: open, answered, dropped. Discuss opens a tutor thread with that question's context.
QU-3	P0	Session openings surface open questions for the current chapter.

5.8 Practice

ID	PRI	REQUIREMENT
PR-1	P1	Generate practice items from covered pages or a session. Types: recall, explain-why, prove or derive, apply, argue. Each stores source pages and concept tags.
PR-2	P1	Review queue with a simple SM-2-style schedule (again, hard, good, easy). Self-graded; tutor grading of a written answer only on request.
PR-3	P1	Grades feed concept status as evidence.

5.9 Weekly targets

ID	PRI	REQUIREMENT
WT-1	P1	Per-subject weekly targets: sessions, minutes, pages or chapter range, practice items reviewed.
WT-2	P1	Progress is computed locally from logs, with no model calls, and shown on the Front Page.
WT-3	P1	Week review on demand: one call summarizes the week and proposes next week's targets for you to accept or edit.

5.10 Learner model

ID	PRI	REQUIREMENT
LM-1	P0	Concepts per subject with status (introduced, shaky, solid), evidence entries, and last-updated time.
LM-2	P0	Updated only from accepted debriefs, practice grades, and your own edits. Never silently mid-session.
LM-3	P0	Learner snapshot: a compact text of at most about 300 tokens, rendered locally from the database, containing relevant shaky concepts, open questions and stated preferences.
LM-4	P1	Stated preferences per subject (e.g. "geometric intuition before formal proof").
LM-5	P0	Every concept and evidence entry can be viewed, edited or deleted.

5.11 Models and usage

ID	PRI	REQUIREMENT
MU-1	P0	Model roles: tutor , fast , deep , vision . Each is a model ID string in settings, with per-subject overrides.
MU-2	P0	Provider adapter with two implementations: Claude Agent SDK signed in with the Claude subscription, and the Claude API with a key. Switchable in settings.
MU-3	P0	Usage log per call: role, model, provider, estimated input and output tokens, latency, success.
MU-4	P0	Usage meter in the status bar (this session, today) with an optional soft warning threshold.
MU-5	P0	Lean mode: smaller context budget, templated openings, fast model for quick actions.
MU-6	P0	On a limit or network error: clear message, the draft message is kept, retry is one click, and the reader keeps working.

5.12 Workspace

ID	PRI	REQUIREMENT
WS-1	P0	Docking layout: Reader, Tutor, Notes, Questions, Session and Outline panes can be resized, collapsed, rearranged and tabbed.
WS-2	P0	Layout presets (Reading, Problem solving, Review), switched automatically by session type (toggle). Custom layouts can be saved.
WS-3	P0	Command palette (Ctrl/Cmd+K) exposes every action.
WS-4	P0	Keyboard shortcuts for all frequent actions (Appendix A).
WS-5	P0	Print (light) and inverted-print (dark) themes; adjustable font size and reading width.

5.13 MCP server

ID	PRI	REQUIREMENT
MC-1	P1	A stdio MCP server exposes study data to Claude Code: read tools plus a few write tools (Section 12).
MC-2	P1	Records created through MCP are marked with source <i>claude-code</i> and appear in the app like any other.

SECTION 06 · BLACK AND WHITE, SET LIKE A NEWSPAPER

UX and UI design

6.1 Design language

Newsprint in black and white. Hierarchy comes from type weight, italics, small caps and rules, never from colour. Newspapers are centuries of refinement in making dense text scannable, which is exactly the job here.

TOKEN	VALUE	USE
ink	#111111	Text, rules, icons
paper	#FAFAF7	Background (warm off-white). Dark theme inverts ink and paper.
grey-1..3	#444 / #777 / #BBB	Secondary text, metadata, disabled states
rule-hair	0.5 px	Pane dividers, table rows
rule-heavy	2 px	Section breaks, active pane top edge
rule-double	2 px + 0.5 px	Masthead and session report header
font-display	Condensed high-contrast serif	Mastheads, screen titles, debrief headline
font-text	Book serif (e.g. Source Serif, Newsreader)	Tutor replies, notes, UI body
font-meta	Mono (e.g. IBM Plex Mono)	Datelines, page refs, timestamps, usage meter, code
radius	0	No rounded corners, no shadows
motion	≤ 120 ms fades	No bouncing, no sliding panels

Fonts are self-hosted in the app so it works offline.

6.2 Screens

SCREEN	PURPOSE AND CONTENTS
Front Page	Home. Masthead with date. Lead story: continue reading (book, page, open session). Columns: This week (targets as rule-drawn progress bars), Open questions, Shaky concepts, Subjects.
Reading Room	The main screen. Docked panes around the PDF. Everything during a session happens here.
Session Report	Close-of-session debrief set like an edition: headline summary, then columns for Covered, Shaky, Questions, Next time. Accept, edit, or discard each item.
Subject Desk	Edit a subject's tutor profile, stated preferences, concepts and weekly targets.
Review Desk	Practice queue, one item at a time, with grading keys.
Settings	Provider, model roles, budgets, lean mode, OCR engine, appearance, data folder, export.

6.3 Default Reading Room layout

+-----+ MARGINALIA Spivak, Calculus Ch.7 Continuity p.143 Problem solving 00:42 +-----+			
OUTLINE (collap- sible)	PDF PAGE	TUTOR	
	[1] <- marginalia pin	p.143 . s7.2 . 14:32	
		Tutor reply set as a short article, serif, no bubbles.	
		"Why can delta depend on x here?" <- your question, italic quote	
		EXPLAIN HINT CHECK ME	
		CHALLENGE SUMMARIZE	
		> ask about p.143 ...	
+-----+			
SESSION: "finish 7.2 exercises" 3 parked Qs usage ~18k tok LEAN off +-----+			

6.4 Tutor panel typography

- **Tutor replies** are set as short articles: a mono small-caps dateline (page, section, time) above serif body text.
- **Your messages** are italic pull quotes with a heavy left rule.
- **Quoted selections** appear as indented blockquotes with a page reference.
- **Quick actions** are small-caps text buttons separated by hairlines, not pill buttons.
- **Marginalia** are numbered like footnotes in the PDF margin and link to their thread.

6.5 Interaction rules

- Nothing modal during reading except the command palette.
- The tutor never interrupts. An optional “stuck nudge” (off by default, P1) offers one line after a set time on one page.
- Everything the model generates for storage (notes, debrief, concept updates, practice) is shown for review before saving.
- Designed empty, loading (“setting type...”), error, offline and limit-reached states for every pane.
- Single-key shortcuts work only when focus is in the reader, so they never fire while typing.

SECTION 07 · LOCAL-FIRST, TYPESCRIPT END TO END

System architecture

Browser (localhost)		Local server (Node, TypeScript)	
+-----+	HTTP/SSE	+-----+	
React UI	<----->	routes/	books sessions tutor notes ...
Reader (PDF.js)		context/	builder, budgets, renderers
Tutor Notes Questions		services/	session learner practice ...
Dock layout + palette		llm/	router -> provider adapter
+-----+			-- agent-sdk (subscription)
			-- anthropic-api (API key)
		ingest/	text layer, outline, OCR
		db/	SQLite (Drizzle)
		+-----+	
			same database
Claude Code	--- stdio --->	packages/mcp (MCP server)	-----+

LAYER	CHOICE	WHY
UI	React, TypeScript, Vite	Fast dev loop, large ecosystem for PDF and docking.
PDF	pdfjs-dist	Standard renderer with text layer and outline access.

LAYER	CHOICE	WHY
Layout	A docking library (e.g. dockview)	Resizable, tabbed, serializable layouts give modularity for free.
Rendering	react-markdown + KaTeX	Maths-heavy subjects need proper LaTeX.
State	Zustand	Small, explicit stores per pane.
Server	Node 20+, Hono or Fastify	Same language as the Agent SDK; streaming via SSE.
LLM	Claude Agent SDK (TypeScript); Anthropic SDK as fallback	Subscription login for personal use; API key if policy or needs change.
Database	SQLite via better-sqlite3 + Drizzle	One file, zero setup, typed schema and migrations.
Schemas	zod in a shared package	Validates structured model outputs (debrief, practice) and API payloads.
OCR	ocrmypdf (Tesseract) as optional CLI; vision via LLM adapter	Free local OCR for prose; vision for maths.
MCP	Official MCP TypeScript SDK, stdio transport	Claude Code launches it as a local tool server.

Agent SDK usage notes

- Pass Marginalia’s own system prompt instead of the Claude Code preset, and disable built-in file and shell tools. This keeps per-turn overhead small and the tutor sandboxed.
- Marginalia owns conversation state and rebuilds context each turn, instead of relying on long-lived SDK sessions. This makes context size predictable and enforceable.
- Custom tools (e.g. *get_page*, *park_question*) can be attached as in-process MCP tools when a turn needs them; v1 needs very few.
- The server binds to localhost only.

SECTION 08 · THE MOST IMPORTANT SECTION

Context engineering and usage optimization

Usage limits are the binding constraint. On a subscription, tutor usage shares a pool with interactive Claude and Claude Code use, and exact quotas are not exposed, so the app tracks estimated tokens and keeps every call lean by construction.

8.1 The context packet for a tutor turn

Ordered from most static to most dynamic, so the stable prefix can be reused by prompt caching where the provider supports it. Budgets are starting values to tune from the usage log.

#	BLOCK	SOURCE	BUDGET (TOKENS)
1	Base tutor instructions	prompts/base.md (static)	~600
2	Subject profile	tutor_profiles (static per subject)	200–400
3	Learner snapshot	Rendered locally from DB, once per session	≤ 300
4	Session header	Book, chapter, session type, goal	≤ 80
5	Trail summary	Rendered locally from events, no model call	≤ 250
6	Conversation	Rolling summary + last 6 turns verbatim	≤ 300 + ≤ 2,500
7	Current page	Cached page text + tail of previous page; or page image	≤ 1,800
8	Selection and message	From the UI	as sent

Target a typical turn of about 6k input tokens, with a configurable hard cap (default 10k). When a turn would exceed the cap, drop in this order: oldest verbatim turns, previous-page tail, trail detail. Output is capped by role (e.g. 1,200 tokens for tutor replies, higher for notes and debriefs).

8.2 Rules

- **No calls on passive events.** Page turns, scrolling and highlighting are logged locally only.
- **Local rendering wherever possible.** The trail summary (“Read pp. 140–143, 12 min on p. 142, 3 highlights, 1 parked question”) and learner snapshot are generated from the database, not by a model.
- **Incremental summaries.** When verbatim turns exceed six, the fast model folds the oldest into the rolling summary in one small call.
- **One call at close.** Debrief, concept updates and notes draft come from a single structured call, validated with zod.
- **On demand only.** Notes, practice items and week reviews are generated only when you ask or accept.
- **Text before images.** Cached text is preferred. Page images are sent only when enabled, downscaled, and their OCR result is cached.
- **Escalate explicitly.** The deep model runs only when you press Go deeper.
- **Measure.** Every call is logged with estimated tokens; a weekly view by role shows where usage goes.

8.3 Model routing defaults

TASK	ROLE	DEFAULT MODEL
Tutor reply, Explain, Hint, Check me, Challenge	tutor	Claude Sonnet 5 (fast model in lean mode for quick actions)
Summarize page	fast	Claude Haiku 4.5
Rolling conversation summary	fast	Claude Haiku 4.5
Session opening line	fast	Claude Haiku 4.5, or a template with no call in lean mode
Session debrief and concept updates	tutor	Claude Sonnet 5
Notes generation, practice generation	tutor	Claude Sonnet 5
Grading a written practice answer	fast	Claude Haiku 4.5
Go deeper (hard proofs, subtle arguments)	deep	Claude Opus 5.5, explicit only
Reading scanned or equation-heavy pages	vision	Claude Sonnet 5 (fast model for plain prose pages)

8.4 Subscription caveats

- Tutor calls through the Agent SDK with subscription login draw from the same plan limits as your other Claude use.
- Anthropic’s policy for subscription-based Agent SDK use changed in 2026 and may change again. The provider adapter isolates this: switching to an API key is a settings change.
- Subscription login is for personal use on your own machine. Distributing the app to others would require each user’s own authentication or API billing.

SECTION 09 · SET AND CHANGE MODELS PER ROLE

Model configuration

Models are addressed by ID strings so new models can be adopted without code changes. Availability depends on your plan and provider; a **Test** button in settings sends a one-line ping per role.

```
{
  "provider": "agent-sdk",           // or "anthropic-api"
  "roles": {
    "tutor": "claude-sonnet-5",
    "fast": "claude-haiku-4-5-20251001",
    "deep": "claude-opus-5-5",
    "vision": "claude-sonnet-5"
  },
  "subjectOverrides": {
    "analysis": { "deep": "claude-opus-5-5" },
    "neuroscience": { "tutor": "claude-sonnet-5" }
  },
  "budgets": { "maxInputTokens": 10000, "maxOutputTokens": { "tutor": 1200, "notes": 3000, "debrief": 2500 } },
  "leanMode": false,
  "usageSoftWarnTokensPerDay": 400000
}
```

SECTION 10 · ONE SQLITE FILE PLUS THE PDFS

Data model

```
~/Marginalia/
marginalia.db           # everything except PDFs
books/<hash>/original.pdf
books/<hash>/ocr.pdf     # only if local OCR was run
exports/notes/...       # Markdown exports
```

TABLE	KEY FIELDS	NOTES
subjects	id, name, slug, tutor_profile_id, created_at	Analysis, Neuroscience, ...
tutor_profiles	id, name, persona, style, answer_policy, verbosity, rules, model_overrides (json), session_type_overrides (json)	Presets seeded from packages/profiles
books	id, subject_id, title, author, file_path, file_hash, page_count, page_offset, text_source, outline (json), last_page	text_source: native ocr_local vision
pages	book_id, page_index, text, text_source, char_count, section_label, needs_ocr	PK (book_id, page_index); the text cache
sessions	id, book_id, subject_id, type, goal, status, started_at, ended_at, start_page, end_page, rolling_summary, debrief (json)	status: active closing closed
events	id, session_id, ts, kind, page_index, dwell_ms, payload (json)	page_view, highlight, note, question, message, action
annotations	id, book_id, page_index, kind, rects (json), quote, note_id, message_id	highlight, underline, bracket, margin_pin
messages	id, session_id, role, action, content, page_index, selection, model, in_tokens_est, out_tokens_est, created_at	action: ask, explain, hint, check, challenge, summarize, deeper
notes	id, subject_id, book_id, session_id, page_from, page_to, concept_ids (json), title, body_md, source, compression	source: user tutor claude-code
questions	id, subject_id, book_id, session_id, page_index, selection, text, status, answer_note_id, created_at, resolved_at	status: open answered dropped
concepts	id, subject_id, name, aliases (json), status, description, last_updated	status: introduced shaky solid
concept_evidence	id, concept_id, session_id, kind, polarity, detail, page_index, created_at	kind: debrief practice user misconception
practice_items	id, subject_id, book_id, page_from, page_to, concept_ids, type, prompt, answer, created_at	

TABLE	KEY FIELDS	NOTES
reviews	id, item_id, ts, grade, interval_days, ease, due_at	SM-2-style scheduling
weekly_targets	id, subject_id, week_start, metric, target	metric: sessions minutes pages items
usage_log	id, ts, session_id, role, model, provider, in_tokens_est, out_tokens_est, latency_ms, ok, error	Feeds the usage meter
settings	key, value (json)	Models, budgets, layouts, appearance

Debrief schema (structured output of the close call)

```
{
  "summary": "Worked through 7.2; proved Theorem 3 with hints; unsure about uniform continuity.",
  "covered": [{ "pages": [140, 143], "section": "7.2" }],
  "concepts": [
    { "name": "uniform continuity", "status": "shaky",
      "evidence": "Treated delta as independent of x on p.142" },
    { "name": "intermediate value theorem", "status": "solid", "evidence": "Explained proof back correctly" }
  ],
  "open_questions": ["Why does a closed interval guarantee uniform continuity?"],
  "next_step": "Problems 7.4 and 7.9, then read 7.3.",
  "notes_draft_md": "optional"
}
```

SECTION 11 · PEDAGOGY AS DATA

Tutor prompt design

Base instructions (skeleton)

You are a tutor sitting beside a student who is reading {book_title}.
You can see the page they are on and a summary of this session. Teach; do not lecture.
Follow the subject profile and answer policy below exactly.
Refer to pages as "p. N". If the page text is unclear or missing, say so instead of guessing.
Keep replies short unless asked for depth. End with at most one question.
Answer policy "hints-first": give the smallest nudge that unblocks the student and offer to go further.
Only write a complete solution or proof when the student says "reveal".

Preset profiles

PROFILE	STYLE	BEHAVIOUR
Analysis	Socratic, hints-first	Asks for exact definitions and quantifiers; proposes counterexamples; checks each proof step; never writes full proofs unless asked to reveal.
Neuroscience	Explainer	Plain language first, then terminology; analogies; links to earlier pages; one check question after explanations.
Philosophy	Interlocutor	Asks you to reconstruct arguments as premises and conclusion; raises objections; flags equivocation; asks you to steelman the opposing view; does not hand you conclusions.
Systems	Explainer + prediction	Walks mechanisms step by step; asks you to predict behaviour before explaining; ties concepts to real code and hardware.

Quick actions are separate prompt files (prompts/actions/*.md) layered on top of the profile, so behaviour can be tuned per action without touching anything else. A small fixed set of test scenarios per profile (about ten) is re-run whenever prompts change, to catch regressions such as over-explaining.

SECTION 12 · YOUR STUDY DATA AS TOOLS

MCP server for Claude Code

Run as `marginalia mcp (stdio)` and register it in Claude Code. It reads the same database as the app.

TOOL	ARGUMENTS	RETURNS / EFFECT
<code>get_current_session</code>	—	Book, page, type, goal, trail summary
<code>get_reading_trail</code>	<code>session_id?</code> <code>days?</code>	Pages, dwell, highlights, questions over the period
<code>get_learner_snapshot</code>	<code>subject</code>	Compact snapshot text
<code>get_concepts</code>	<code>subject</code> , <code>status?</code>	Concepts with status and latest evidence
<code>get_notes</code>	<code>subject?</code> , <code>book?</code> , <code>concept?</code> , <code>since?</code>	Notes as Markdown
<code>get_open_questions</code>	<code>subject?</code> , <code>book?</code>	Open parked questions with page refs
<code>search_pages</code>	<code>book</code> , <code>query</code>	Matching page snippets from the local text cache
<code>add_note</code>	<code>subject</code> , <code>title</code> , <code>body_md</code> , <code>book?</code> , <code>pages?</code>	Creates a note (source: claude-code)
<code>add_practice_items</code>	<code>subject</code> , <code>items[]</code>	Adds items to the review queue
<code>park_question</code>	<code>subject</code> , <code>text</code> , <code>book?</code> , <code>page?</code>	Adds an open question

Example requests in Claude Code: “Turn this week’s open Spivak questions into a problem set,” “What did I struggle with in systems this month?”, “Write a one-page summary of my philosophy notes on Hume.”

SECTION 13 · ONE REPO, CLEAR SEAMS

Project structure and modularity

```
marginalia/
  apps/
    web/                                React UI
      src/
        panes/                          reader/ tutor/ notes/ questions/ session/ outline/
        layout/                         dock config, presets (reading, problem, review)
        screens/                        front-page/ reading-room/ session-report/ subject-desk/ review-desk/ settings/
        components/                     typographic primitives: Masthead, Rule, Dateline, PullQuote, SmallCapsButton
        commands/                       command registry + keyboard shortcuts
        state/                          zustand stores
        api/                            typed client (shared zod schemas)
        theme/                          tokens.css, self-hosted fonts
      server/                           local Node server
        src/
          routes/                       books sessions tutor(SSE) notes questions practice targets settings usage
          services/                     session learner practice targets
          context/                       builder.ts budgets.ts trail.ts snapshot.ts
          llm/                           provider.ts agent-sdk.ts anthropic-api.ts router.ts usage.ts
          prompts/                       base.md debrief.md notes.md practice.md actions/*.md
          ingest/                         text-layer.ts outline.ts ocr-worker.ts
          db/                             schema.ts migrations/
        packages/
          shared/                        zod schemas and types (debrief, settings, events, api)
          db/                             Drizzle client used by server and MCP
          mcp/                           stdio MCP server
          profiles/                       preset tutor profiles (json)
        data/                             dev data directory (gitignored)
```

Extension points

TO ADD	YOU WRITE	YOU REGISTER
A pane	One component in panes/	Pane registry entry: id, title, component, default size
A quick action	One prompt file in prompts/actions/	Action registry entry: id, label, shortcut, role, output type
A tutor preset	One JSON file in packages/profiles/	Seeded automatically
A model provider	One class implementing LLMProvider	Provider map in llm/router.ts
An OCR engine	One function implementing extractPageText	Engine option in book settings
An MCP tool	One handler in packages/mcp	Tool list in the server definition

```
interface LLMProvider {
  stream(req: LLMRequest): AsyncIterable<LLMChunk>; // tutor replies
  complete<T>(req: LLMRequest, schema: ZodType<T>): Promise<T>; // debrief, practice, notes
}
interface LLMRequest {
  role: "tutor" | "fast" | "deep" | "vision";
  system: string; messages: Msg[]; images?: PageImage[]; maxOutputTokens: number;
}
```

SECTION 14 · BUILD ORDER

Milestones and acceptance criteria

MILESTONE	CONTENTS	DONE WHEN
M0 Reader foundation	Import, library, PDF.js reader, page and dwell tracking, text layer detection, OCR paths, page text cache, base theme	A born-digital and a scanned PDF both have text for every page; events are logged; no model calls exist yet.
M1 Tutor	Tutor pane, context builder with budgets, highlight-to-ask, quick actions, provider adapter, model settings, usage log and meter	Replies correctly reference the current page; typical turns stay under budget; switching a role's model in settings takes effect.
M2 Sessions and memory	Open and close ritual, Session Report review, concepts and evidence, parked questions, learner snapshot, tutor profiles and presets	Closing produces a reviewable debrief; the next session's opening references it; hints-first is honoured in Analysis.
M3 Workspace	Docking layout, presets per session type, command palette, shortcuts, marginalia pins, Front Page, dark theme, focus mode	A full session can be run without the mouse.
M4 Extras (P1)	Notes generation and export, practice and review queue, weekly targets, MCP server	Condensed notes for a session in one action; review queue works; Claude Code can list open questions.
Dogfood week	Use it daily across at least two subjects	A written friction log decides v1.1.

SECTION 15 · MEASURED ON ONE USER: YOU

Success metrics

METRIC	SIGNAL
Study sessions per week vs target	Does the tool make starting and sustaining sessions easier?
Share of sessions closed with an accepted debrief	Is the session ritual worth doing?
Times you left the app to look something up (self-reported)	The core promise: fewer focus breaks.
Open questions resolved within a week	Is parking questions actually leading to answers?
Practice recall rate over time	Is material being retained?

METRIC	SIGNAL
Estimated tokens per study hour	Is usage staying within your plan comfortably? Should trend down as budgets are tuned.

SECTION 16 · WHAT COULD GO WRONG

Risks and open questions

RISK	MITIGATION
Subscription policy for Agent SDK changes	Provider adapter plus API-key fallback; model roles let cheaper models absorb most calls.
Poor OCR on maths	Vision engine per book or page; math-aware converters (e.g. Marker) as a later engine.
Tutor over-explains or gives away answers	Answer policy in profile, action-specific prompts, fixed regression scenarios per profile.
Wrong concept labels in the learner model	Nothing saves without review; everything is editable and deletable.
Latency breaks flow	Streaming, fast model for quick actions, small context packets.
Scope creep	This document's non-goals; features enter only through the friction log.

Open questions

- Should concepts be free-form names proposed by debriefs, or seeded per chapter from the outline?
- Should the debrief generate automatically on close, or only when you press Close session?
- Is local Tesseract setup acceptable on your OS, or should Vision be the default OCR path?
- Is a desktop wrapper (e.g. Tauri) worth it after v1, for file access and a single-click launch?

SECTION 17 · REFERENCE

Appendix: shortcuts and glossary

A. Keyboard shortcuts

KEYS	ACTION	KEYS	ACTION
Ctrl/Cmd+K	Command palette	Q	Park a question
A	Ask about selection	N	New note
E	Explain	S	Summarize page
H	Hint	F	Focus mode
C	Check me	1 / 2 / 3	Layout preset
X	Challenge	Ctrl/Cmd+Enter	Send message
Ctrl/Cmd+Shift+D	Go deeper (deep model)	Ctrl/Cmd+.	Close session

Single-key shortcuts are active only when focus is in the reader.

B. Glossary

TERM	MEANING
Reading trail	The ordered log of pages, dwell times, highlights, notes, questions and messages in a session.
Trail summary	A short text rendered locally from the trail and included in tutor context.
Learner snapshot	A compact, locally rendered summary of relevant concepts, open questions and preferences.
Debrief	The structured end-of-session report, reviewed before it updates the learner model.

TERM	MEANING
Marginalia	Tutor replies pinned to a page and shown in its margin.
Parked question	A question saved for later with its page and selection attached.
Role	A named model slot (tutor, fast, deep, vision) mapped to a model ID in settings.
Lean mode	A setting that shrinks context and routes quick actions to the fast model to save usage.